

▼ Repaso

▼ Scripts

- ☒ Ejercicio 1
- ☒ Ejercicio 2
- ☒ Ejercicio 3

- Control de errores

▼ Procedimientos

- ☒ Ejercicio 4
- ☒ Ejercicio 5
- ☒ Ejercicio 6
- ☒ Ejercicio 7
- ☒ Ejercicio 8
- ☒ Ejercicio 9
- ☒ Ejercicio 10
- ☒ Ejercicio 11
- ☒ Ejercicio 12

▼ Funciones

▼ Funciones integradas en el SGBD

- ☒ Ejercicio 13
- ☒ Ejercicio 14
- ☒ Ejercicio 15
- ☒ Ejercicio 16
- ☒ Ejercicio 17

▼ Funciones propias

- ☒ Ejercicio 18
- ☒ Ejercicio 19
- ☒ Ejercicio 20
- ☒ Ejercicio 21

Repaso

Scripts



Resumen scripts

Definir variables de usuario

```
SET @cod=105, @cad='Hola';
```

Mostrar valores de variables

```
SELECT @cod, @cad;
```

Calcular y asignar valor en la misma instrucción

```
SELECT @numclientes:= COUNT(*) FROM CLIENT;
```

Almacenar datos (sin mostrar)

```
SELECT COUNT(*) INTO @numreg FROM CLIENT;
```

Consultar todas las variables creadas por el usuario (depende de la versión)

```
SELECT * FROM information_schema.USER_VARIABLES;
```

Recuerda

Tipo de variable	Ejemplo	Dónde se usa
Variable de usuario	@x	consultas normales
Variable local	DECLARE x INT	procedimientos

✓ Ejercicio 1

Crea un script que asigne a una variable el valor 10 y a otra el valor 20. Muestra el resultado de la suma de ambas variables.

✓ Ejercicio 2

Crea un script que obtenga el número total de jugadores de la tabla jugadores y lo almacene en una variable.

✓ Ejercicio 3

Crea un script que obtenga el número de filas de cada tabla de la base de datos nba y visualice el resultado en una tabla con dos columnas: nombre de la tabla y número de filas.

Control de errores

Control de errores

Sintaxis

```
DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN
    [instrucciones]
END;
```

Comprobar errores

```
-- Mostrar errores
SHOW WARNINGS;

-- Almacenar el código y el mensaje
GET DIAGNOSTICS;
```

Recuerda el `HANDLER` debe declararse antes de las instrucciones que pueden fallar.

Procedimientos

Resumen procedimientos

Sintaxis

```
DELIMITER //
```

```
CREATE PROCEDURE nombre()  
  BEGIN  
    instrucciones;  
  END //
```

```
DELIMITER ;
```

Llamada

```
CALL nombre();
```

Bloques (agrupan instrucciones)

```
BEGIN  
  ...  
END
```

Eliminar procedimientos

```
DROP PROCEDURE IF EXISTS datos_cliente;
```

Consultar procedimientos almacenados en la base de datos

```
SHOW PROCEDURE STATUS;
```

Variables locales al procedimiento

- Tipos: *Los mismos que se emplean en el DDL*
- Declaración `DECLARE variable tipo`
- Asignación de valores `SET variable = valor`
- Valor por defecto `DEFAULT`

Parámetros de entrada **IN**. Si no indicamos nada, los parámetros por defecto son de entrada

```
CREATE PROCEDURE entrada(p1 INT) ...  
-- Se puede pasar un valor  
CALL entrada(5);  
-- O una variable de usuario  
CALL entrada(@valor);
```

Parámetros de salida **OUT**. Devuelven un valor al finalizar el procedimiento

```
-- Para recibir el valor de salida, se debe usar siempre una variable de usuario (@)
CREATE PROCEDURE salida(p1 INT, OUT p2 INT) ...;
CALL salida(5, @resultado);
```

Parámetros de entrada/salida **INOUT**.

- Se lee como dato de entrada.
- El procedimiento puede modificar ese valor.
- El valor modificado se devuelve al terminar.

```
-- Para recibir el valor se debe usar siempre una variable de usuario (@)
CREATE PROCEDURE contar(INOUT cuenta INT, incremento INT) ...;
CALL contar(@cantidad, 5);
```

Estructuras de control

```
IF condicion THEN sentencia
  [ELSEIF condicion THEN sentencia] ...
  [ELSE sentencia]
END IF
```

```
CASE condicionQueTomaValor
  WHEN valor THEN sentencia
  [WHEN valor THEN sentencia] ...
  [ELSE sentencia]
END CASE
```

```
CASE
  WHEN condicionEvaluada THEN sentencia
  [WHEN condicionEvaluada THEN sentencia] ...
  [ELSE sentencia]
END CASE
```

```
WHILE condicion DO
  sentencias
END WHILE
```

✓ Ejercicio 4

Modifica el procedimiento datos_jugador para que reciba como entrada el código de un jugador.

```

DELIMITER //
DROP PROCEDURE IF EXISTS datos_jugador //
CREATE PROCEDURE datos_jugador()
BEGIN
    DECLARE num_temporadas INT;
    DECLARE total_puntos DECIMAL(10,2);

    SELECT COUNT(es.temporada), SUM(es.Puntos_por_partido)
    INTO num_temporadas, total_puntos
    FROM estadisticas es
    WHERE es.jugador = 106;

    SELECT j.codigo, j.Nombre, num_temporadas, total_puntos
    FROM jugadores j
    WHERE j.codigo = 106;
END //
DELIMITER ;

```

✓ Ejercicio 5

Crea un procedimiento denominado `media_puntos_equipo`, que reciba como parámetro el nombre de un equipo y devuelva en una variable la media de puntos por partido de los jugadores de ese equipo.

Muestra un mensaje de error si no se puede calcular la media.

✓ Ejercicio 6

Crea un procedimiento que calcule una predicción de puntos por partido de un jugador tras una mejora de rendimiento.

El procedimiento se denomina `prediccion_puntos` y recibe como parámetro la media de Puntos_por_partido de un jugador, devolviendo en el mismo parámetro la predicción de puntos tras aplicar un incremento del 10% (se devuelve la media de Puntos_por_partido incrementada en un 10%). Realiza un control de errores.

✓ Ejercicio 7

Calcular el bonus de un jugador

Crea un procedimiento denominado `bonus_jugador`, que devuelva el bonus de un jugador dado su identificador de jugador (`codigo`) y una temporada (`temporada`).

El bonus se calcula como un porcentaje de sus `Puntos_por_partido` según la siguiente tabla:

- Si los puntos por partido son menores que 10, el bonus es del 5%.
- Si los puntos por partido están entre 10 y 20, el bonus es del 10%.
- Si los puntos por partido son mayores que 30, el bonus es del 20%.

El procedimiento debe mostrar por pantalla el nombre del jugador, la temporada, sus puntos por partido y el bonus que ha obtenido. No se realiza ningún cambio en la base de datos.

Si el código del jugador no existe o no tiene estadísticas en la temporada indicada, se mostrará el mensaje *Jugador no encontrado o sin estadísticas*.

✓ Ejercicio 8

Actualizar el peso de un jugador

Crea un procedimiento que actualice el `Peso` de un jugador dado su identificador (`codigo`) y un porcentaje de incremento.

El procedimiento debe realizar las siguientes acciones:

- Si el porcentaje de incremento es negativo, mostrar un mensaje de error y no realizar ninguna actualización.
- Si el nuevo peso es superior a 150, mostrar un mensaje de advertencia indicando que el peso es muy alto.
- Si el nuevo peso es superior a 150, Actualizar el `Peso` del jugador en la tabla `jugadores`.
- Mostrar por pantalla el nombre del jugador, su nuevo peso y el nombre de su equipo.

Si el jugador no existe, se mostrará el mensaje *Jugador no encontrado*.

✓ Ejercicio 9

Actualizar el equipo de un jugador

Crea un procedimiento que actualice el equipo de un jugador dado su identificador (`jugadores.id_jugador`) y el nombre del nuevo equipo (`equipos.nombre_equipo`).

El procedimiento debe realizar las siguientes acciones:

- Comprobar si existe el jugador y el equipo.
- Actualizar el campo `equipos.nombre_equipo` del jugador con el nombre del nuevo equipo indicado.

✓ Ejercicio 10

Mostrar los jugadores de un equipo.

Crea un procedimiento que muestre por pantalla los jugadores de un equipo dado su nombre (`equipos.nombre_equipo`), posición (`jugadores.posicion`) y un peso mínimo (`jugadores.peso`).

El procedimiento debe realizar las siguientes acciones:

- Mostrar el nombre de los jugadores del equipo cuyo peso mínimo sea superior al indicado, ordenando el resultado por peso de mayor a menor y, en caso de empate, por nombre en orden ascendente.
- Si no existe el equipo o la posición o el peso mínimo es inferior a 0, mostrar un mensaje indicando el error y finalizar el proceso.

✓ Ejercicio 11

Contar jugadores por posición

Crea un procedimiento que muestre el número de jugadores que hay en cada posición (`jugadores.posicion`).

El procedimiento debe realizar las siguientes acciones:

- Mostrar cada posición y el número total de jugadores que tiene.
- Agrupar los resultados por la posición.
- Ordenar el resultado por número de jugadores de mayor a menor.
- En caso de empate, ordenar alfabéticamente por la posición.

✓ Ejercicio 12

Peso medio de jugadores por equipo

Crea un procedimiento que muestre el peso medio de los jugadores de cada equipo (`equipos.nombre_equipo`).

El procedimiento debe realizar las siguientes acciones:

- Calcular el peso medio de los jugadores para cada equipo.
- Mostrar el nombre del equipo y su peso medio.
- Agrupar los resultados por equipo.
- Mostrar solo los equipos cuyo peso medio sea mayor que un valor indicado (`peso_medio_min`).
- Ordenar el resultado por peso medio de mayor a menor.

Funciones

Funciones integradas en el SGBD



Funciones integradas

Funciones matemáticas

Función	Descripción
ABS(x)	Valor absoluto de x.
FLOOR(x)	Parte entera inferior de x.
MOD(x,y)	Resto de la división x / y.
POW(x,y)	x elevado a y.
SQRT(x)	Raíz cuadrada de x.
RAND()	Número aleatorio entre 0 y 1.
ROUND(x,d)	Redondea x con d decimales.
ROUND(x)	Redondea x al entero más cercano.
SIGN(x)	1 si x>0, -1 si x<0, 0 si x=0.

Funciones de cadenas

Función	Descripción
CONCAT(c1,c2,...)	Concatena varias cadenas.
UPPER(c)	Convierte la cadena a mayúsculas (UCASE).
LOWER(c)	Convierte la cadena a minúsculas (LCASE).
LEFT(c,k)	Devuelve los k primeros caracteres.
RIGHT(c,k)	Devuelve los k últimos caracteres.
SUBSTRING(c,k,l)	Subcadena desde posición k con longitud l.
SUBSTRING_INDEX(c,pt,k)	Parte de la cadena según aparición del patrón pt.
INSTR(c,c2)	Posición donde aparece c2 en c (0 si no aparece).
LENGTH(c)	Número de caracteres de la cadena.
TRIM(c)	Elimina espacios al inicio y final.
REPEAT(c,k)	Repite la cadena k veces.
REPLACE(c,cb,cr)	Sustituye cb por cr en la cadena.
REVERSE(c)	Invierte la cadena.
STRCMP(c1,c2)	Compara cadenas (-1,0,1).

Funciones de fechas

Función	Descripción
CURDATE()	Fecha actual.
CURTIME()	Hora actual.
NOW()	Fecha y hora actual.
ADDDATE(f,d)	Suma d días a la fecha f.
ADDTIME(t,s)	Suma s segundos al tiempo t.
DATEDIFF(f1,f2)	Días entre f1 y f2.
TIMEDIFF(t1,t2)	Diferencia entre tiempos t1 y t2.
DAY(f)	Día del mes de la fecha f.
MONTH(f)	Mes de la fecha f.
YEAR(f)	Año de la fecha f.
DAYOFWEEK(f)	Día de la semana (1=domingo).
DAYNAME(f)	Nombre del día de la semana.
HOUR(t)	Hora del tiempo t.
MINUTE(t)	Minutos del tiempo t.
SECOND(t)	Segundos del tiempo t.
SEC_TO_TIME(s)	Convierte segundos a tiempo.
TIME_TO_SEC(t)	Convierte tiempo a segundos.
MAKETIME(h,m,s)	Crea un tiempo con h,m,s.
STR_TO_DATE(cf,ff)	Convierte texto a fecha.
DATE_FORMAT(f,ff)	Formatea una fecha como texto.

Funciones para mostrar información

Función	Descripción
VERSION()	Versión de MySQL
DATABASE()	Base de datos
CURRENT_USER()	Usuario
CONVERT(parm, tp)	Convierte parm al tipo tp
ISNULL(var)	1 si var es NULL, 0 en caso contrario
IFNULL(parm, msg)	Si parm es NULL retorna msg si no retorna parm

✓ Ejercicio 13

Peso medio de los jugadores de un equipo

Crea un procedimiento que devuelva el peso medio (*AVG*) de los jugadores de un equipo (`jugadores.peso`) redondeado a dos decimales (*ROUND*).

El procedimiento recibe como parámetro el nombre del equipo (`equipos.nombre_equipo`).

✓ Ejercicio 14

Crea un procedimiento que devuelva el factorial de un número entero. El procedimiento recibe como parámetro el número entero y el resultado se devuelve en un parámetro de salida. Si el número es negativo se devolvera -1.

✓ Ejercicio 15

Generar código de jugador

Crea un procedimiento que dado el identificador de un jugador (`jugadores.id_jugador`) muestre un código compuesto por las tres primeras letras de su nombre (`jugadores.nombre`), un guión y su identificador.

En el caso de un jugador 23, el código generado sería MIC-23.

Funciones: *CONCAT* y *LEFT*.

✓ Ejercicio 16

Crea un procedimiento, que reciba como argumento una fecha en formato yyyy-mm-dd y devuelva en otro parámetro la fecha en formato 'día de la semana, día de mes, mes, año con cuatro dígitos (ejemplo: lunes, 12 de enero de 2025).

Función: *DATE_FORMAT*

✓ Ejercicio 17

Estadísticas de un jugador

Crea un procedimiento que dado el identificador de un jugador (`jugadores.id_jugador`) devuelva los puntos máximos, mínimos y medios (`estadisticas.puntos`).

El procedimiento recibe como parámetro el identificador del jugador y devuelve los valores en tres parámetros de salida.

Funciones: *MAX* , *MIN* y *AVG*.

Funciones propias



Resumen funciones

Sintaxis

```
CREATE FUNCTION nombre (parámetros) RETURNS tipo
BEGIN
    bloque con RETURN
END;
```

Llamada

```
SELECT nombre(parámetros);

-- Uso
SELECT precioConIVA(1234);

-- Consulta de una tabla con un campo calculado
SELECT COM_NUM, TOTAL AS Neto, precioConIVA(TOTAL) AS 'Total con Iva' FROM COMANDA;
```

```
DROP FUNCTION IF EXISTS nombre;
```

✓ Ejercicio 18

Nombre del equipo

Crea una función que, dado un jugador (`jugadores.id_jugador`) devuelva el nombre de su equipo

Llama a la función desde un SELECT que muestre el nombre del jugador y el nombre del equipo, ordenado por equipo.

✓ Ejercicio 19

Verificar si un jugador existe

Crea una función que verifique si un jugador existe en la base de datos, dado su identificador (`jugadores.id_jugador`).

La función debe devolver un valor booleano (1 si existe, 0 si no existe).

✓ Ejercicio 20

Estadísticas de un jugador

Crea una función que reciba un código de jugador (`jugadores.id_jugador`) y devuelva el número de registros de estadísticas que tiene un jugador, dado su identificador.

Ejecuta la función desde un SELECT que muestre el nombre del jugador, el número de estadísticas y el total de puntos.

✓ Ejercicio 21

Calcular la antigüedad de un jugador.

Crea una función que calcule la antigüedad de un jugador en años, dado su identificador (`jugadores.id_jugador`).

La función debe devolver un valor numérico que represente la cantidad de años transcurridos desde la fecha de ingreso (`jugadores.fecha_ingreso`) del jugador.

Utiliza *YEAR()* y *CURDATE()*.

Ejecuta la función desde un **SELECT** que muestre el nombre del jugador, la fecha de ingreso y la antigüedad en años.